PROJECT REPORT: personalized information retrieval

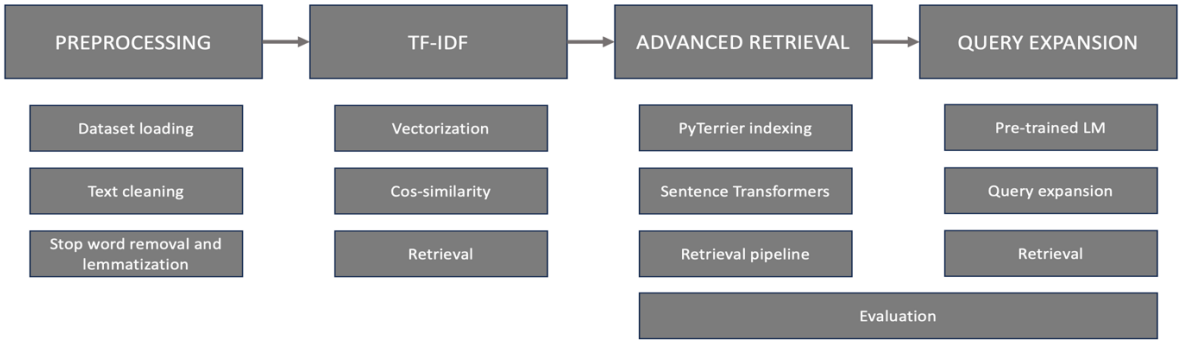
Information retrieval and recommender systems – a.y. 2024/25

INTRODUCTION AND INTUITIONS

Team: Leonardo Castelli (512919), Francesca Chite (513011), Chiara Bosi (513029).

Aim of the project: developing a search engine (SE) to retrieve relevant answers to user queries from SE-PQA dataset. The SE integrates personalization based on contextual features of the queries.

General structure: the notebook exploits different techniques of information retrieval (IR), from simpler approaches to more advanced and cutting-edge strategies; our initial goal is to determine the optimal retrieval method by comparing offline evaluations of the same subset of queries over different embedding strategies, and our final goal is to provide the user with more tailored answers by enhancing the ranked retrieval with query expansion based on contextual features.
It follows a schematic overview of the project.



After preprocessing the data, the project illustrates the implementation of tf-idf for basic information retrieval; then, enhanced techniques are developed and compared to tf-idf using Mean Average Precision (MAP), Normalized Discounted Cumulative Gain (NDCG), and Precision at 10 (P@10). Finally, the best retrieval pipeline is applied to the expanded queries in order to assess whether the integration of context empowers retrieval.

The main goal of IR is to retrieve all relevant documents with respect to users' queries, while minimizing the number of non-relevant documents. This kind of measurement of the effectiveness is done offline, following **Cranfield Paradigm**: given a corpus, a set of queries and relevance judgments (the ground truth), the effectiveness of the model is evaluated by comparing the retrieved documents with qrels.
This comparison is done using different evaluation metrics: **Mean Average Precision** (MAP), **Normalized Discounted Cumulative Gain** (NDCG), and **Precision at 10** (P@10). MAP focuses on the average precision across all relevant documents for each query, and then averages among all queries; P@10 measures the precision of the top 10 retrieved results for each query; NDCG compares rankings to an ideal order where all relevant items are at the top of the list.

Preprocessing is the initial step aimed at removing redundancies and noise in the data, preparing documents to be transformed in more structured representations. Typical cleaning pipelines include tokenization, stop-words, URLs, HTML tags and special characters removal and lemmatization; each is applied to both the corpus and the queries to ensure that their embeddings can be compared.
Once standardized, simplified the text and extracted the tokens, **indexing** mechanisms are applied to efficiently store the obtained vocabulary. For this project, PyTerrier's inverted file structure has been adopted.
An inverted index is a data structure composed of a dictionary (containing all unique terms of the corpus) and a posting file (storing document identifiers and the positional reference of the token in the document).

One of the baseline approaches for information retrieval is **TF-IDF**, a measure of the importance of a word according to its frequency and its relevance in the corpus (according to Zipf's law and Luhn's analysis, the most meaningful words are neither the rarest, nor the most common). This is achieved by combining TF, which measures word frequency within the document, and IDF, which minimizes the weights of common words and emphasizes those of rare words across a corpus. As a result, higher scores are obtained for terms that are frequent in one document but rare overall.

The Vectorization process generates a numerical vector for each document, creating a vocabulary where every entry represents a word's TF-IDF score. To identify the most relevant documents for each query, the cosine similarity is computed for every possible query-document pair within the dataset. This operation measures the cosine of the angle between the two vectors (the representations of the query and the document) to indicate the similarity between the two representations.

More advanced embedding strategies involve **Bi-Encoders** and **Cross-Encoders**. These methods leverage neural networks to capture the semantic relationships between words and concepts, performing deeper and more sophisticated embeddings. Both encoders use pre-trained models from the sentence-transformers library to generate embeddings: numerical vector representations that capture the semantic meaning of text.
Bi-Encoders, preferable for their speed, process queries and documents independently, producing separate embeddings for each and then compares them using cosine similarity to assess relevance. In contrast, Cross-Encoders, more accurate but also computationally expensive, evaluate query-document pairs jointly by encoding them together, enabling the model to assign a direct relevance score to each pair.

Query expansion enriches the user's initial query by incorporating additional terms or phrases that reflect their preferences and interests. This step is performed before encoding, ensuring that the enriched query fully captures the user's information need (context) and provides a more comprehensive input to the Bi-Encoder and Cross-Encoder. Contextual features like tags are fundamental to enhance personalization; incorporating this kind of information to the query expansion language model allows to point it in the direction of users' needs, enabling the model to generate new tokens which are assumed to be coherent with the information need and the characteristics of the user. The retrieval system can benefit from these features by integrating query expansion mechanisms using pre-trained LLMs. Our intuition is that the more complete the query is, the more appropriate the answers are.

METHODOLOGY

As anticipated, the notebook is divided into 4 parts: preprocessing, tf-idf exploration, advanced retrieval, retrieval with query expansion.

PREPROCESSING includes the analysis of the dataset and the development of cleaning, stop-word removal and lemmatization function.

Analysis of the dataset. The folder contains what follows:

- ***subset_answers***, a subset of answers which will constitute the corpus of the project (the documents to be retrieved) composed by the tuples document identifier (*docno*) and text of the answer (*text*).
- ***val_queries*** and ***test_queries***, two different subsets of queries used for validation and testing (in particular, the validation set has been adopted for the choice of the indexing pipeline, and the test set has been used to compare the personalized retrieval with and without enhanced features, as will be discussed later) composed by the query identifier (*quid*), the textual content of the query (*query*), the identifier of the author of the query (*user*_id), a list of tags associated to the query (*tags*)….
- ***val_qrels*** and ***test_qrels***, telling which documents (*docno*) are considered relevant to which query (*qid*) in the validation/test data.

Note: due to limitations in computational power, the query dataset has been drastically reduced. If possible, it is suggested to run the notebook with the entire data. The following results will probably be affected by the lack of queries.

Preprocessing functions.

- ***clean_text()*** function includes the removal of URLs, HTML tags, special characters and punctation, numbers and extra whitespaces. Both the corpus and the queries undergo this pipeline.
- ***lemmatizer_with_stopwords()*** function integrates both stop-words removal and lemmatization (both assessed by adopting nltk library set for English vocabulary). This method is used for both the corpus and the queries only for tf-idf exploration; it is not adopted for more advanced indexing/embedding strategies.

TF-IDF EXPLORATION shows how to employ tfd-idf for basic retrieval. The aim of this section is not to provide an evaluation of the method (which will be assessed later in comparison with more advanced techniques), but simply to demonstrate the implementation pipeline and how to retrieve documents adopting this mechanism.
After cleaning the text of the queries and the corpus (in order to uniform their content to ensure better consistency for the next steps), TfidfVectorizer is used to create a TF-IDF representation of the corpus. Note: only the first 500 terms are considered for faster computations. These steps produce a matrix displaying the weights of each term based on its frequency and relevance (rarity, discriminatory power) in the documents. It is, then, computed the cosine similarity between the representation of a query and that of the documents in the corpus. This operation returns a list of scores which determines the final ranking of the retrieved documents, among which the top 10 are selected. The dataframe of the top-n (10, by default) ranked retrieved documents given the test queries is stored and, at the end of the project, it will be compared with the results obtained by implementing different strategies.

ADVANCED RETRIEVAL employs more advanced indexing and embedding techniques. The aim of this section is to compare different pipelines in order to assess the optimal (best performing) strategy according to Mean Average Precision (MAP), Normalized Discounted Cumulative Gain (NDCG), and Precision at 10 (P@10) evaluation metrics.

1. The corpus is indexed using ***PyTerrier***'s *IterDictIndexer*.
2. Queries are transformed by using *sentence_transformers* to generate embeddings using ***cross-encoding*** (for comparing query-document pairs) and ***bi-encoding*** (for encoding queries and documents separately).
3. The PyTerrier retrieval pipeline includes tf-idf, cross-encoding, bi-encoding and a combination of each of the last two with tf-idf (50% each after being normalized).
4. ***MAP, NDCG***, and ***P@10*** are evaluated over the (query) validation set in order to determine the highest performing model.
5. The best model is, then, evaluated over the test set and it is performed a ranked retrieval of the n (10, by default) most relevant documents with respect to each query. The procedure is iterated to all the queries present in the set; the resulting dataframe is stored and it will be compared and discussed later.

QUERY EXPANSION, as anticipated, is expected to help retrieving more appropriate documents with respect to the user's queries by enhancing the textual content of the queries themselves through a pre-trained LM (trained on similar data: natural language queries).

- It is adopted ***AutoModelForCausalLM*** pre-trained on *microsoft/Phi-3-mini-4k-instruct* and optimized for GPU use.
- The ***query_expansion*** function receives as input the original query and some keywords (the **tags** of the query), it prompts the LM to generate an expanded query that incorporates user interests (according to the keywords) and returns the extended query. The procedure is iterated through all the test queries.
- The extended query dataset undergoes the same retrieval and evaluation method adopted in the previous paragraph with the best pipeline identified before (pt. 5 of "Advanced retrieval").

EXPERIMENTS AND RESULTS

As discussed before, due to limitations in computational power, tests were conducted on heavily reduced subsets of queries. The notebook has been tested twice: once on a subset of 50 queries, and once on 250 queries. More accurate results would be produced by inspecting the entire query dataset.
The following tables summarize the results obtained on the validation test (used to determine the best performing pipeline) and the ones obtained on the test set (before and after query expansion).

	name	map	ndcg	P.10
0	TF_IDF	0.653394	0.715161	0.082
1	BiEnc	0.805000	0.839562	0.094
2	.5*BiEnc + .5TF_IDF	0.727382	0.778359	0.088
3	CrossEnc	0.763056	0.804067	0.090
4	.5*CrossEnc + .5TF_IDF	0.690336	0.747908	0.088

Validation on 50 queries

	name	map	ndcg	P.10
0	TF_IDF	0.735349	0.775378	0.0844
1	BiEnc	0.847189	0.865247	0.0916
2	.5*BiEnc + .5TF_IDF	0.809604	0.835743	0.0892
3	CrossEnc	0.775301	0.808094	0.0880
4	.5*CrossEnc + .5TF_IDF	0.780498	0.812138	0.0876

Validation on 250 queries

	name	map	ndcg	P.10
0	BiEnc	0.851667	0.867869	0.09

Test on 50 queries

	name	map	ndcg	P.10
0	BiEnc	0.847284	0.864857	0.0908

Test on 250 queries

	name	map	ndcg	P.10
0	BiEnc	0.804583	0.827986	0.088

Test on 50 + query expansion

	name	map	ndcg	P.10
0	BiEnc	0.816315	0.838078	0.0892

Test on 250 + query expansion

In the pictures above, it is possible to compare the results obtained for the three metrics in 6 cases:

- validation on 50 and 250 queries for all defined pipelines
- test for the BiEncoder only on 50 and 250 queries
- test for the BiEncoder only on 50 and 250 expanded queries.

As a general trend, it can be noticed that, the smaller the sample, the higher the variability.

The tests unanimously determined BiEncoder pipeline as the best performing pipeline for PyTerrier embedding and retrieval. BiEncoder consistently outperformed the TF-IDF and CrossEncoder pipelines in terms of MAP, NDCG, and P@10 metrics. BiEncoders are better than TF-IDF at capturing deeper semantic relationships and they are faster than CrossEncoders as they allow for precomputing document embeddings, which is an advantage in particular for large datasets.

	qid	user_id	docid	docno	query	text	rank
0	academia_185177	1532620	10928	academia_185179	after what george was georgetown university na...	georgetown university is named after the villa...	0
1	academia_185177	1532620	10800	academia_16379	after what george was georgetown university na...	here is one side effect of a university having...	1
2	academia_185177	1532620	6176	history_45209	after what george was georgetown university na...	san andreas was named after its church the fir...	2
3	academia_185177	1532620	10888	academia_69225	after what george was georgetown university na...	so focusing on the uk to start the universitie...	3
4	academia_185177	1532620	5724	judaism_98419	after what george was georgetown university na...	according to the rambam the kohen gadol pronou...	4

	qid	user_id	docid	docno	query	text	rank
0	academia_185177	1532620	10928	academia_185179	after whom was george washington university na...	georgetown university is named after the villa...	0
1	academia_185177	1532620	6136	history_38222	after whom was george washington university na...	no england would not have been called england ...	1
2	academia_185177	1532620	10800	academia_16379	after whom was george washington university na...	here is one side effect of a university having...	2
3	academia_185177	1532620	6176	history_45209	after whom was george washington university na...	san andreas was named after its church the fir...	3
4	academia_185177	1532620	1661	scifi_86624	after whom was george washington university na...	in the deadpool and death annual we learn the ...	4

Ranked retrieved documents before and after query expansion

The final goal of the experiment is to compare the metrics scores resulted from the application of the encoder on the original test sets, with those obtained from the application on the expanded ones. It is clear from the output of the evaluation that the encoder suffers a decrease in its performance, especially on the smaller test sample. However, the scores do not drop significantly for any metrics. Often, the first retrieved document stays the same; what typically changes may be the order of the other retrieved documents and, in some cases, only a few are completely discarded and replaced.

CONCLUSIONS

We believe that our results are pretty sound, but they would be even more accurate if the datasets were not reduced that much due to the limited computational resources of Colab. In fact, a simple comparison of the evaluation metrics in the case of 50 and of 250 queries proved that larger samples yield more accurate and representative results. Actually, a wider query set would cover a broader range of topics and relevance patterns, allowing the system to generalize better among different contents and be less prone to random variation, averaging out such fluctuations.

It might strike our attention that there has been a decrease in the scores of the precision metrics on the test set after query expansion. Some considerations might be underlined: the possible over-expansion of the queries may have introduced irrelevant or noisy terms and lead to the retrieval of less relevant documents (in some cases, it might have even deviated from the original user's objective of the question, causing a semantic mismatch); or, in contrast, the improvement in document ranking due to query expansion may have affected the model's threshold adopted to classify the relevance of a document, leading to the disregard of the less relevant ones (considered irrelevant, in contrast with the relevance judgments qrels).

In conclusion, while we still judge our findings to be robust, we are sure that future work leveraging more extensive datasets and resources could further enhance the system's accuracy and generalizability. Additionally, exploiting or collecting more contextual features and integrating them into the retrieval and ranking processes could push personalization and precision to even higher levels.